2.Doris|Starrocks表设计与操作面试题

由 涤生大数据管理员创建, 最终由 涤生老师-阳哥修改于 九月 12, 2026

s

- 1.表设计相关
  - 1.1 Doris中Unique数据模型和Duplicate数据模型的区别（度小满）
  - 1.2 Doris/StarRocks 的分区和分桶有什么区别，怎么去选择？（必看，极其高频面试题）
  - 1.3 Doris/StarRocks 的分区策略及其优化方式是什么？（悦智）
  - 1.4 Doris/Starrocks 里面的列存是如何设计的？相对行存来说，有什么区别?
  - 1.5 Bitmap 索引与Bloom filter 索引适用场景与限制（微创）
  - 1.6 Doris/Starrocks 的表模型有哪些？分别适用于哪些场景？（必看，极其高频面试题）
  - 1.7 Doris的unique 表是如何实现合并的?
  - 1.8 doris bitmap索引实现方式（华盛通/阿里小鹿中医/飞轮科技/京东健康）
  - 1.9 Doris/Starrocks表应该如何设计？你从哪几个方面考虑？（跨越速运/壹向科技）
  - 1.10 doris/starrcoks是如何创建前缀索引的？（跨越速运）
  - 1.11 doris有几种聚合模式？（豆神科技）
  - 1.12 Doris有哪些索引，该如何选择（必看，极其高频面试题）
  - 1.13 doris 建设中key怎么划分，比如2个key，一个高基数列，一个低基数列，怎么做（跨越速运）
  - 1.14 doris复合索引的字段顺序影响索引效果吗？（跨越速运）
  - 1.15 doris unique 表模型的MOW 和MOR的区别和优化点 （流利说/稳健医疗/极兔速递/金碧AI研究院）
  - 1.15 doris中最小的存储单位是什么？ （杭州织光映画科技）
  - 1.16 Doris 的自动分区和动态分区有什么区别？（上海蓝标）
    - 1. 动态分区（Dynamic Partition）
    - 2. 自动分区（Auto Partition）
  - 1.17 介绍一下Doris 自增列以及与mysql的区别（上海蓝标）
- 2. 数据操作相关
  - 2.1 Doris和starrocks 如何实现数据更新？（上海径赢）
  - 2.2 介绍下Doris/Starrocks的导入方式？（KK深圳/中经社/搜狐/度小满/小米/预言家/多乐游戏/上海径赢/道通科技）
  - 2.3 Doris/Starrocks如何保证导入的事务及原子性？（北京花径教育科技 ）
  - 2.3 Doris Stream Load原理浅析（度小满）
  - 2.4 Doris 的Routine Load实现原理（度小满）
  - 2.5 Doris 的Broker Load实现原理（度小满）
  - 2.6 Doris 的Mysql Load实现原理（度小满）
  - 2.7 Doris-connector 数据一致性的实现
  - 2.8 介绍下Doris数据导入的compaction机制（杭州织光映画科技）
  - 2.9 什么是Doris的单副本导入机制?
  - 2.10 介绍下Doris Flink Connector 2PC 实现原理
  - 2.11 Doris如何实现表结构的变更
  - 2.12 Doris如何进行数据导出?
  - 2.13 介绍下Doris中的Sequence列
  - 2.14 Doris/StarRocks 中的列存储和行存储有何区别及应用场景?
  - 2.15 doris明细模型的分区的更新是怎么做的？我要回溯一下、数据更新一下的话，是先删再写吗？（面试题）
  - 2.16 在doris/starrocks，如何实现数据的全量和增量导入?
  - 2.17 讲下Doris表关联物理关联的几种方式（跨越速运/深圳艾特尔/杭州织光映画科技/上海蓝标/metshort）
  - 2.18 doris如何强制使用broadcast？（跨越速运/上海蓝标）
  - 2.19介绍下 Doris/Starrocks SQL 解析流程
  - 2.20 实时项目中kafka到doris同步方式是怎样的？ kafka到doris同步 有几种方式？（趣致）
  - 2.21 公司有用doris的部分列更新吗？怎么使用的/适用什么场景（深圳万邦速达）

1.表设计相关

1.1 Doris中Unique数据模型和Duplicate数据模型的区别（度小满）

使用上：

- Unique会根据用户指定的key将相同的key进行更新替换掉（查询只显示最新的数据）
- Duplicate不管用于指定的key是什么，都会进行持续的追加，旧的数据不会被更新替换（明细模型）

原理上：

- Duplicate模型就是可以简单的理解为是一个 append only的明细表，数据完全按照导入文件中的数据进行存储，不会有任何聚合。即使两行数据完全相同，也都会保留。这种数据模型适用于既没有聚合需求，又没有主键唯一性约束的原始数据的存储。
- Unique模型主要是保证key的唯一性，新插入的数据会基于主键会进行一步的合并操作，达到显示新数据的要求。Unique模型有读时合并（merge on read)和写时合并（merge on write）这种实现方式（1.2之后引入，性能更好）。其目的都是为了保证查询时候读出来的是最新的数据。

1.2 Doris/StarRocks 的分区和分桶有什么区别，怎么去选择？（必看，极其高频面试题）

分区是**更高层级的逻辑划分**，分桶是**分区内部的物理存储划分**（即分桶是分区内的细分）。分桶会将分区后的每段数据打散为逻辑分片Tablet。

划分依据：

- 分区：基于业务逻辑字段（如时间、地域）。
- 分桶：基于高基数字段（如用户 ID）+ 哈希算法。

核心作用：

- 分区：快速定位数据区间，减少全局数据扫描量。
- 分桶：让数据在分区内均匀分散，提升并行查询效率。

选择与结合策略

- **选分区表**：数据量大，需频繁按时间、地域等字段快速过滤数据（如实时时序数据查询）。
- **选分桶表**：数据含高基数字段，需并行处理多维度查询（如用户行为的并发分析）。

注意：单个 Tablet（分桶）（**Tablet 数 = 分区数 * 桶数 * 副本数**）的数据量理论上没有上下界，除小表（百兆维表）外需确保在 **1G - 10G** 的范围内：

- 如果单个 Tablet 数据量过小，则数据的聚合效果不佳，且元数据管理压力大。
- 如果数据量过大，则不利于副本的迁移、补齐，且会增加 Schema Change 或者 物化 操作失败重试的代价（这些操作失败重试的粒度是 Tablet）。

1.3 Doris/StarRocks 的分区策略及其优化方式是什么？（悦智）

StarRocks 的分区策略是将数据根据特定的列（分区键）值分散存储到不同的分区中。这样做可以优化查询性能，原理和方法如下：

1.分区键选择： 选择合适的分区键是优化查询性能的关键。通常选择经常用作查询条件的列作为分区键，可以有效减少查询时需要扫描的数据量。

2.分区范围： 根据业务需求合理设定分区范围，如按照时间范围（如日、月、年）或其他业务逻辑划分，以确保数据分布均匀，避免某些分区过大而影响查询性能。

3.分区方式：可以手动分区，也支持动态/自动分区功能，可以自动创建和删除分区，简化分区管理，并保证数据分布的实时性和均衡性。

4.分区裁剪： 在执行查询时，会根据查询条件对分区进行裁剪，只扫描包含相关数据的分区，减少数据扫描量，加速查询响应。

1.4 Doris/Starrocks 里面的列存是如何设计的？相对行存来说，有什么区别？

（1）Doris / StarRocks 列存设计要点

- **列式物理布局**：同一列的值连续存放，按列分 page/segment 写入与读取，便于只读所需列，减少 I/O。数据以 page 为最小编码/压缩单元，多个 page 组成 segment/文件，文件尾存元数据和索引，读取按需加载。
- **编码与压缩**：列级采用多种编码（plain, delta, dictionary 等）和压缩，提高压缩比与 CPU 解压效率（同类型数据压缩更好）。
- **多种内置索引**：ZoneMap（min/max）、短键/前缀索引（Short-Key/Prefix）、Ordinal、以及可选的 Bitmap / BloomFilter 等，用来快速跳过大量无关 page/file，减少扫描。
- **写入路径：MemTable→Flush→Segment**：写到内存表后批量转为列式文件并生成索引（短键、zone map 等）；这是列存高吞吐的关键。

（2）列存和行存关键区别

1. **访问模式**
   - 列存：适合**宽表的分析型负载**（聚合、扫描、少列选择）；读取少量列但大量行时优势巨大。
   - 行存：适合**点查/事务/单行写多**（OLTP），单行所有列一次性读写开销低。
2. **I/O 与压缩**
   - 列存：同类型数据压缩率高，磁盘带宽利用率好；但读取 `SELECT *` 或很多列时会产生对每列的独立 I/O。
   - 行存：压缩较差，但单行读写仅需一次 I/O（特别在小宽度表上更优）。
3. **CPU/执行模式**
   - 列存：向量化处理、谓词下推、早期投影（列裁剪），CPU 利用更高。
   - 行存：逐行处理更简单，适合事务语义与复杂行内操作。
4. **更新与写入延迟**
   - 列存：写入通常为批量（MemTable flush），随机写/频繁行级更新代价高（需写新version + 后台compaction）。
   - 行存：支持原地更新，延迟低（适合高并发写/更新）。
5. **索引与跳过扫描**
   - 列存依赖 ZoneMap/short-key/bloom/bitmap 等索引来大幅减少扫描量（这是列存仍能做高效点查/过滤的手段）。

1.5 Bitmap 索引与Bloom filter 索引适用场景与限制（微创）

（1）Bitmap 索引：为每个取值构建一个 bit 位图（这一列第 N 行是否等于该值），多个位图可按 AND / OR 快速组合，适合高效过滤与聚合。

适用场景：

- 低基数列（如性别、状态码， distinct 值≤1000），因位图存储效率随基数升高急剧下降；
- 精确等值查询（如`where status=1`）及逻辑组合查询（`and/or`），通过位图位运算快速计算结果。

限制：

- 高基数列不适用（如用户 ID），会导致位图数量激增，存储和维护成本过高；
- 不适合频繁更新场景（如实时写入），单条记录更新需修改多个位图，性能损耗大。

（2）Bloom Filter 索引：基于哈希函数构建位数组判断“值是否存在”，存在一定概率误判，但不会漏判。

适用场景：

- 高基数列（如订单号、URL），无需存储精确信息，仅需快速判断 “值是否可能存在”；
- 作为前置过滤（如`where id=xxx`），减少无效数据扫描，常用于查询的早期阶段。

限制：

- 存在假阳性（误判 “存在”），不能用于精确查询，需配合后续验证；
- 不支持范围查询（如>, <）和排序；
- 低基数列性价比低（低基数更适合 Bitmap），且哈希函数设计影响准确性。

1.6 Doris/Starrocks 的表模型有哪些？分别适用于哪些场景？（必看，极其高频面试题）

Doris 有Duplicate/Aggregate/Unique三种模型，Starrocks 比它多一个Primary Key，下面介绍这些表模型的机制和适用场景

（1）明细表（Duplicate Key）

- **机制**：按指定列排序存储，数据完全保留（不聚合、不去重），支持任意维度查询。
- **适用场景**：原始数据存储（如用户行为日志、设备埋点数据），需完整保留明细且查询维度灵活的场景。

（2）聚合表（Aggregate Key）

- **机制**：按「聚合键」预聚合数据（如 sum、count、max 等），相同聚合键的新数据会合并旧数据。
- **适用场景**：统计分析场景（如 PV/UV 统计、销售汇总），需高频查询汇总结果，且能容忍数据预聚合的场景。

（3）更新表（Unique Key）

- **机制**：按「唯一键」去重，新数据覆盖旧数据（支持部分列更新），保证唯一键对应的数据唯一。
- **适用场景**：需高频更新部分字段的场景（如订单状态更新、用户信息修改），更新维度固定且无需保留历史版本。

（4）主键表（Primary Key）（starrocks特有）

- **机制**：支持主键级别的插入、更新、删除，提供行级事务一致性，数据按主键有序存储。

- **适用场景**：强事务性需求（如实时业务库同步）、需保留最新状态且高频更新的场景（如用户画像、库存数据）。

核心差异：明细表强调 “完整保留”，聚合表强调 “预计算”，更新表 / 主键表强调 “数据更新”（主键表事务性更强）。

1.7 Doris的unique 表是如何实现合并的?

Doris 的 Unique 表是类 **LSM-Tree（Log-Structured Merge-Tree） 存储结构**，数据写入时都是以追加方式生成新的版本文件，不直接修改已有文件，保证高写入性能。然后依靠 Compaction 来完成数据去重与文件合并。

Compaction 会在后台自动运行，把同一个分片下的多个小文件读取出来，按照主键进行合并，**相同主键只保留最新版本的数据**，旧版本数据会被清理掉。

合并完成后会生成一个完整的大文件，同时删除原来的小文件，这样既能保证主键唯一性，又能减少文件数量，提升查询效率，还能回收存储空间。

Unique 表有MOW（ Merge-on-Write）和MOR（Merge-on-Read）两种 模式，这两种模式的主要区别就是数据写入后合并的时机，MOW是写入的时候就进行合并了，保证了查询时候的性能。MOR是读取的时候才会合并数据。

1.8 doris bitmap索引实现方式（华盛通/阿里小鹿中医/飞轮科技/京东健康）

Doris 的 Bitmap 索引是一种**跳过索引（Skip Index）**，适用于**中等基数**（建议 100~100,000 之间）的列，如城市、职业等字段。

实现原理：为每个不同的值维护一个 bit 数组，每一位对应一行数据，1 表示该行包含该值，0 表示不包含。查询时通过 **位运算（AND/OR/NOT）**快速定位满足条件的行，跳过不相关的数据块，减少 I/O。

- 只支持**等值查询和 IN 查询**，不支持范围查询
- 只能建在**单列**上
- 适合正交查询（多条件组合过滤）场景，位运算效率极高

1.9 Doris/Starrocks表应该如何设计？你从哪几个方面考虑？（跨越速运/壹向科技）

Doris 表设计主要围绕**查询性能、导入效率、集群均衡、运维成本四个方面**，从**数据模型、分区、分桶、索引、字段类型、键类型**这几个方面设计：

1. 选合适数据模型：明细用 Duplicate，更新用 Unique，聚合用 Aggregate；
2. 合理分区：按时间分区，控制单分区数据量，便于冷热分离，删除和按照分区裁剪数据；
3. 合理分桶：按高基数、查询频繁的维度分桶，避免数据倾斜，控制分桶数，充分利用BE并发能力；
4. 设计键与排序键：将过滤、Join 字段设为前缀索引，提升查询效率；
5. 索引选择：低基数列建 Bitmap，高基数用 BloomFilter，按照其他需求合理选择索引，减少不必要索引降低写入开销。；
6. 字段与类型：精简字段，使用小数据类型，减少存储与 IO。

1.10 doris/starrcoks是如何创建前缀索引的？（跨越速运）

Doris 的前缀索引**无需手动创建**，是建表时自动生成的内置索引，不过需要注意使用方式：

1. **自动创建**：建表时，Doris 自动取 Key 列的前 **36 字节**作为前缀索引，无需任何额外语法。
2. **VARCHAR 截断规则**：遇到 VARCHAR 类型列时，前缀索引立即截断，后续列不再纳入，且 VARCHAR 最多取前 20 字节。
3. **一表只有一个**：由于 Key 定义唯一，每张表只有一套前缀索引，因此建表时应将**最高频的过滤列放在 Key 的最前面**。
4. **变通方案**：若某列不在前缀索引中但需要加速，可以：① 创建其他索引；② 对 Duplicate 表创建**物化视图**调整列顺序，间接实现多前缀索引。

1.11 doris有几种聚合模式？（豆神科技）

Doris 的聚合模型（Aggregate Key Table）支持以下几种聚合方式：

1. **SUM**：累加多行的值，适合求总量（如总消费、总点击数）
2. **REPLACE**：用后批次数据替换之前的值
3. **MAX / MIN**：保留最大值或最小值

4. **REPLACE_IF_NOT_NULL**：仅替换非 NULL 值，NULL 不覆盖原值
5. **HLL_UNION**：对 HLL 类型列使用 HyperLogLog 算法聚合，用于近似去重
6. **BITMAP_UNION**：对 BITMAP 类型列做位图并集聚合，用于精确去重

如果以上聚合方式不满足业务需求，还可以使用 `AGG_STATE` 类型配合 `state/merge/union` 函数组合器实现自定义聚合逻辑。

1.12 Doris有哪些索引，该如何选择（必看，极其高频面试题）

Doris 的索引分两大类：

内置索引（自动维护，无需手动创建）

- **前缀索引**：基于有序存储，自动取 Key 列前 36 字节，适合最高频的等值/范围过滤
- **ZoneMap 索引**：自动维护每列的 Min/Max/Null 统计，用于跳过不满足条件的数据块

二级索引（手动创建）

- **倒排索引**：适用最广，支持等值、范围、全文检索（MATCH），支持多条件 AND/OR/NOT 组合，首选推荐
- **BloomFilter 索引**：适合高基数列（>5000）的等值/IN 查询，存储空间中等，不支持范围查询
- **NGram BloomFilter 索引**：专门加速字符串 LIKE 查询
- **Bitmap 索引**：适合中等基数（100~100,000）列的等值查询，支持正交多条件位运算

选择建议（三步走）：

1. 最高频过滤列 → 放 Key 最前面，利用**前缀索引**
2. 非 Key 列需要加速 → 优先建**倒排索引**（通用性最强）
3. 特殊场景补充：LIKE 查询加 **NGram BloomFilter**；存储空间敏感时用 **BloomFilter** 替代倒排索引

1.13 doris 建设中key怎么划分，比如2个key，一个高基数列，一个低基数列，怎么做（跨越速运）

1. 先分清：这两列在业务里怎么用？

- 谁经常出现在 `WHERE` / `JOIN ON` 里？
  更常用来 缩小扫描范围 的列，一般更适合 靠前（前缀索引、min/max 裁剪更容易帮到它）。
- 谁几乎不参与过滤？
  放后面往往更合理，否则前缀对多数查询帮助小。

所以：高基数 vs 低基数是次要信息，「过滤频率 + 选择性」更重要。

2. 高基数、低基数一般评估

情况	评估
高基数且常做等值/范围过滤（如 user_id、订单号）	靠前有利于 点查/小范围 时跳过大量无关数据（仍要结合是否真有这类 SQL）。
低基数但几乎每次必带（如 `dt`、国家）	若绝大多数查询都带 `dt`，`dt` 在前有时整段数据都更好裁（尤其再配合 分区）。
高基数但随机、很少单独过滤	放最前不一定划算，前缀对「只按另一列查」帮助有限。

3. 和分区、模型的关系

- 分区键（如按天）解决「删整段、裁大窗」；Key 顺序解决 tablet 内扫描裁剪，两者配合看真实 SQL。
- AGG / UNIQUE：Key 定义的是 聚合粒度 / 主键语义，不能只按「查得快」乱定，要先满足 业务唯一性 / 聚合维度，再在同一约束下微调顺序。

1.14 doris复合索引的字段顺序影响索引效果吗？（跨越速运）

Doris 复合索引（前缀索引）的字段顺序对索引效果有决定性影响。Doris 在建表时，会自动取 Sort Key 的前 36 字节（遇到varchar还会截断的）作为前缀索引（Prefix Index）。这是一个稀疏索引，每隔 1024 行记录一个索引条目。前缀索引只对从最左列开始连续命中的列有效，而且前缀索引拼接 Sort Key 列时，遇到 VARCHAR 列立即截断，后续列不再进入前缀索引；所以创建表的时候，应该将高频过滤列、固定长度类型的列放在 Sort Key 的最前面，VARCHAR 列尽量靠后。

1.15 doris unique 表模型的MOW 和MOR的区别和优化点 （流利说/稳健医疗/极兔速递/金碧AI研究院）

区别

维度	Merge-on-Write (MoW)	Merge-on-Read (MoR)
合并时机	写入时	查询时或 Compaction 时
写入行为	查找旧行并在 delete bitmap 中标记删除，写入新行	仅追加，保留所有版本
查询性能	高，接近 Duplicate Key 表	低，查询时需实时合并多版本，约慢 3-10 倍
写入性能	略低（小批量约低 10-20%，大批量约低 30-50%）	高，接近 Duplicate Key 表
资源消耗	写入和 Compaction 消耗更多 CPU/内存	查询消耗更多 CPU/内存
适用场景	读多写少（推荐，2.1 起默认）	写多读少（不再推荐）
默认版本	**2.1 起默认**	需手动指定

MoW 工作原理

MoW 每次 upsert 经历 5 个阶段：

1. **缓冲批次**：BE 将数据收集到 memtable 并按 Key 排序
2. **查找旧行**：通过 per-segment 主键索引定位已有行的 rowset 和 row ID
3. **标记删除**：在 delete bitmap 中将旧行标记为已删除（数据仍在磁盘，等待 Compaction 清理）
4. **Sequence 列仲裁**：若设置了 `sequence_col`，保留 Sequence 值更大的行（解决乱序写入）
5. **发布新 rowset**：事务提交，新版本对查询可见，查询通过 delete bitmap 过滤旧行

MoR 工作原理

写入时只追加数据，保留每个 Key 的所有版本。查询时实时合并多版本，返回最新值。随着版本积累（两次 Compaction 之间），查询延迟会持续增长。

MoW 相比 MoR 额外支持以下特性：

特性	MoW	MoR
部分列更新（Partial Column Update）	✅ 支持	❌ 不支持
Filter 下推到存储层	✅ 支持	❌ 不支持
count(*) 高效查询	✅ 比 Aggregate 模型快 10 倍以上	❌ 需合并版本，开销大
读时无版本合并	✅	❌

选型建议

- **绝大多数场景选 MoW**：实时更新、CDC 同步、维度表、报表查询、需要部分列更新等
- **仅在写远多于读且需最小化写入开销时考虑 MoR**：但官方已标注"不再推荐"

1.15 doris中最小的存储单位是什么？ （杭州织光映画科技）

在 Apache Doris 中，最小的存储和管理单位根据视角的不同有以下划分：

- **逻辑与管理层面：Tablet（数据分片）** 是 Doris 中最小的逻辑存储单元和数据管理单元，也是副本调度、数据迁移与复制的基本物理单位。每个 Bucket（分桶）在物理上对应一个 Tablet 。
- **物理底层文件与压缩层面：**
  - **Segment**：每个 Tablet 由多个 Rowset 组成，Rowset 内部由 **Segment 文件**（通常上限为 256 MB）存储实际列存数据。
  - **Page**：在 Segment 文件内部，每列数据被切分为约 64 KB 的 **Page**。**Page 是 Doris 列式存储中进行编码、压缩/解压以及 Zone Map 统计信息的最小物理单位**

完整的存储层次关系为：`Table → Partition → Tablet (Bucket) → Rowset → Segment → Column → Page`。

1.16 Doris 的自动分区和动态分区有什么区别？（上海蓝标）

在 Apache Doris 中，**动态分区（Dynamic Partition）** 和 **自动分区（Auto Partition）** 都是简化分区维护的自动化特性，其中自动分区是动态分区的演进替代方案

两者的核心区别如下：

主要区别对比

比较维度	动态分区 (Dynamic Partition)	自动分区 (Auto Partition)
分区创建时机	由系统后台线程基于**时间调度规则**周期性创建	**数据写入时按需实时创建**（有新数据到来时才触发建分区）
分区列依赖	强依赖**系统当前时间**，必须是与系统时间相关的时间列	**与系统时间解耦**，可处理历史数据、非连续离散时间或非时间类型
支持的分区类型	仅支持单列的 **Range 分区**（`DATE` / `DATETIME`）	支持 **AUTO RANGE**（`DATE`, `DATETIME`）和 **AUTO LIST**（多列及多种枚举类型如 `VARCHAR`, `INT` 等）
生命周期管理 (TTL)	通过 `dynamic_partition.start` 按时间偏移量自动清理过期历史分区	**AUTO RANGE 原生支持**通过 `partition.retention_count` 保留指定数量的历史分区
未来分区处理	需配置 `dynamic_partition.end` 提前预建未来分区以防写入失败	无需预建，未来分区在写入数据时自动生成，且不受保留策略影响
典型适用场景	依赖固定时间滚动周期的实时日志、监控数据	离线历史数据补录、分区值离散不可预测、多租户枚举列表以及通用分区管理

核心特性详解

1. 动态分区（Dynamic Partition）

- **工作机制**：按固定时间周期（天、周、月等）滚动维护分区
- **生命周期配置**：
  - `dynamic_partition.start`：指定保留的历史分区范围，早于该范围的分区会被自动删除
  - `dynamic_partition.end`：提前预创建未来若干周期的分区
- **限制**：只适用于单列时间 Range 分区，且对历史回补或远期超出预建范围的数据不够灵活

2. 自动分区（Auto Partition）

- **工作机制**：建表时不预建空分区，数据写入时根据数据具体的值实时、按需自动创建对应分区
- **生命周期配置**：
  - 在 AUTO RANGE 表中，通过配置 `"partition.retention_count" = "N"`，系统会**仅保留分区值最大的 N 个历史分区**（即上界小于等于当前时间的分区），而当前和未来的分区不受影响全量保留

1.17 介绍一下Doris 自增列以及与mysql的区别（上海蓝标）

1. Doris 如何设置自增列?

在 Doris 中建表时，可以通过在列定义上添加 `AUTO_INCREMENT` 属性来设置自增列

- 如果不指定起始值，默认起始值为 `1`
- 可以使用 `AUTO_INCREMENT(start_value)` 手动指定起始值（必须大于或等于 0）

2. 约束限制与 MySQL 的区别

Doris 的自增列在设计上有以下约束与差异：

1. **表模型限制**：仅支持在 **Duplicate 模型** 和 **Unique 模型** 表中包含自增列（Aggregate 模型不支持）

2. **列类型与数量限制**：
   - 一张表**最多只能包含一个**自增列
   - 自增列必须是 **`BIGINT`** 类型，且必须为 **`NOT NULL`**
3. **用户指定非空值不会校验唯一性（与 MySQL 的重要区别）**：
   - **不包含或写入 `NULL`**：Doris 会自动生成表级别唯一的递增值
   - **用户显式写入非 `NULL` 值**：Doris 会保留用户写入的值，但**不会校验或强制保证用户指定值的唯一性**，这可能会破坏自增列的唯一性并导致重复数据

3. Doris 的自增 ID 是连续的吗?

不一定绝对连续，且不保证时间先后顺序

Doris 生成的自增值通常是**稠密**的，但存在以下特性：

- **可能存在空洞（Gaps）**：为了提升分布式写入性能，每个 BE（Backend）节点会预先分配一段独立的、互不重叠的唯一值区间（Block），因此在不同节点分配或导入时可能出现 ID 不连续的空洞
- **非按时间单调递增**：Doris 无法保证后写入的数据生成的自增值一定大于先写入的数据，因此**不能使用自增 ID 来推断数据写入的时间先后顺序**

2. 数据操作相关

2.1 Doris和starrocks 如何实现数据更新？（上海径赢）

在 Doris 和 StarRocks 中，数据更新不是直接修改磁盘数据，而是通过 **追加写入新版本 + 逻辑覆盖（或删除标记） + 后台合并** 实现的。

1. 底层机制统一是：写新版本，不改旧数据

- 更新或删除数据时，会写入一条新记录或删除标记（Delete Bitmap）。
- 查询时通过版本号 + 删除标记合并出最新结果。
- 后台通过 Compaction 清理旧版本，真正释放空间。

2. 不同表模型支持更新方式不同

（1）在 Doris 中

- **表模型决定更新能力**：三类表 — **主键（Unique Key）/聚合（Aggregate Key）/明细（Duplicate Key）**。主键模型是支持实时 UPSERT/部分列更新的核心；聚合表通过聚合函数（如 REPLACE）实现按 key 合并；明细表为纯追加。
- **两种合并策略**：写时合并（MoW，默认且推荐）与读时合并（MoR）。MoW 在写入时做去重合并，查询快；MoR 写入快但查询需在线合并，开销大。
- **更新/删除实现**：
  - **UPSERT（导入）**：主键表通过导入（Stream/Broker/INSERT 等）实现 UPSERT，遇到相同主键覆盖旧行。
  - **UPDATE/DELETE（DML）**：执行时会扫描满足条件的行，然后写回新行或写入删除标记（`__DORIS_DELETE_SIGN__`），查询时过滤，物理回收由后台 compaction 完成。`DELETE` 在不同模型下实现（Delete Sign / Delete Predicate）。

（2）在 StarRocks 中

- **Primary Key 表（推荐用于高频更新）**：主键表设计有主键索引 + DelVector（删除向量），写入时加载主键索引到内存：对 Update 操作做 **Delete（标记旧行） + Insert（写新行）**，并更新主键索引以指向最新数据文件与行号，查询直接读取最新记录，无需在线合并，查询性能高，适合用 CDC 实时同步事务型数据。
- **Unique / Update 表（历史上用于更新）**：以唯一键 + `_version` 语义实现“多版本取最新”的逻辑（REPLACE），写入按批次产生版本号，查询取版本号最大的行；该表类型目前逐步被 Primary Key 表替代。
- 其他与Doris类似

2.2 介绍下Doris/Starrocks的导入方式？（KK深圳/中经社/搜狐/度小满/小米/预言家/多乐游戏/上海径赢/道通科技）

（1）导入本地数据可以通过stream load或者mysql load的方式

（2）从外部存储导入（例如：hdfs以及所有支持s3存储协议的对象存储），使用hdfs load或者s3 load的方式。

（3）Kafka中的消息数据，可以通过routine load以近实时的方式进行数据传输。

（4）外部表中的数据可以通过catalog或者flinkcdc的形式进行同步。（x2doris也支持离线的数据同步）。

starrocks类似，不过没有x2doris

2.3 Doris/Starrocks如何保证导入的事务及原子性？（北京花径教育科技）

Doris 中的所有导入操作都有原子性保证，即一个导入作业中的数据要么全部成功，要么全部失败。不会出现仅部分数据导入成功的情况。Doris 的导入作业都可以设置一个 Label。这个 Label 通常是用户自定义的、具有一定业务逻辑属性的字符串。Label 的主要作用是唯一标识一个导入任务，并且能够保证相同的 Label 仅会被成功导入一次。Label 机制可以保证导入数据的不丢不重。如果上游数据源能够保证 At-Least-Once 语义，则配合 Doris 的 Label 机制，能够保证 Exactly-Once 语义。

2.3 Doris Stream Load原理浅析（度小满）

详细原理过程地址：https://doris.apache.org/zh-CN/blog/principle-of-Doris-Stream-Load

Stream Load是Doris用户最常用的数据导入方式之一，它是一种同步的导入方式, 允许用户通过Http访问的方式批量地将CSV格式或JSON格式的数据导入Doris，并返回数据导入的结果，除此之外，也可以通过在客户端执行查询SQL来查询历史任务的结果。另外，Doris还为Stream Load提供了操作审计功能，可以通过审计日志对历史的Stream Load任务信息进行审计。

Stream Load具体执行流程如下：

1. 用户提交Stream Load的Http请求到FE，FE对Http的Header解析（其中包括解析数据导入的库、表、Label等信息），然后进行用户鉴权。如果Http的Header解析成功并且用户鉴权通过，FE会将Stream Load的Http请求转发到一台BE节点，该BE节点将作为本次Stream Load的Coordinator；否则，FE会直接向用户返回Stream Load的失败信息。
2. Coordinator BE接收到Stream Load的Http请求后，会首先进行Http的Header解析和数据校验，其中包括解析数据的文件格式、数据body的大小、Http超时时间、进行用户鉴权等。如果Header数据校验失败，会直接向用户返回Stream Load的失败信息。
3. 当上述数据校验都通过后，FE会生成任务生成导入计划，并且开启事务向Coordinator BE进行导数，Coordinator BE接受到数据之后会通过BRPC同步至Executor BE，当Executor BE完成数据导入后，Coordinator BE通过Thrift RPC 向FE发送Commit Transaction的请求。
4. FE收到Coordinator BE发送的Commit Transaction的请求之后，会对事务进行提交， 并向Executor BE发送 Publish Version的任务，如果Executor BE正常完成FE返回Commit Transaction，如果执行超时，FE返回Publish Version，最终由Coordinator BE向用户返回Stream Load的最终结果。

2.4 Doris 的Routine Load实现原理（度小满）

Routine Load使得Doris可以提供一个常驻的导入任务，可以不断的从指定的数据源读取数据，将数据导入到Doris中。其底层是基于Stream load方式导入实现的。作为一个常驻形式的导入任务，Routine Load的每一次导入都会看成一个Task，每个Task 由FE发送给BE执行, Task完成一批数据导入后通知FE，每批次的决定因素有最大超时时间/最大读取的条数/最大读取的一批的大小，FE更新Offset后继续下发新Task，不断重复这个过程，从而完成数据写入。

其实现原理流程如下：

（1）FE 通过 JobScheduler 将一个导入作业拆分成若干个 Task。每个 Task 负责导入指定的一部分数据。Task 被 TaskScheduler 分配到指定的 BE 上执行。（task数量并不是无上限的，其主要是Doris集群任务的并发数/槽位数决定）

（2）在 BE 上，一个 Task 被视为一个普通的导入任务，通过 Stream Load 的导入机制进行导入。导入完成后，向 FE 汇报。

（3）FE 中的 JobScheduler 根据汇报结果，继续生成后续新的 Task，或者对失败的 Task 进行重试。

（4）整个 Routine Load 作业通过不断地产生新的 Task，来完成数据不间断的导入。

优点：

高效性：Routine Load 使用分布式架构和并行处理技术，能够充分利用集群的计算和存储资源，以快速、高效地导入大规模数据。

可扩展性：由于采用了分布式设计，Doris Routine Load 具有良好的可扩展性，可以根据需要增加节点，以应对不断增长的数据导入需求。

注意事项

1. 本质上还是调用的stream load任务，我们需要注意攒批的频次的设置，不能太过于频繁的去调用，避免未合并的版本数超限。

2. routine load任务的消费频次可以根据上有的数据量变化做一个设定。根据消息的峰值变化速率来设定不同任务的消费频次。
3. 过多的routine load任务会占用一定的硬件资源，会导致查询性能的下降。（进程常驻）

2.5 Doris 的Broker Load实现原理（度小满）

核心利用Broker进程拉去远端存储的数据，同BE做相关的数据交互。

流程详解

1. 用户在客户端创建broker load任务。
2. FE在接收到用户的创建请求后，会根据请求导入的源文件的数据量和文件数量以及BE的数量生成PLAN，并将plan分发到多个be节点上，每个be会负责一定数据量的导入任务。
3. 对应的be在接收到导入任务时，会通过broker进程，去远端存储上拉去相应的数据到对应的be上。
4. 所有的数据拉去结束后，由FE判断导入任务是否成功，并返回最终结果给用户

注意事项

1. 最好是每个BE的节点上创建一个broker进程，同时broker进程的名称保持一致，用户在发起导入任务的时候，可以尽可能的保证所有的broker进程和对应的BE节点参与到导入任务中来，最大化的提高导入性能。
2. 源文件不建议是数量较多的碎片化小文件，同时也不建议是数据量比较大的数量较少的文件个数，可以酌情把小文件合并或者大文件拆分成数量为BE倍数的个数，单个文件大概在数十GB大小的文件。
3. 可以通过提升并行度来提升导入速度，但是注意资源使用以及compaction的压力情况

2.6 Doris 的Mysql Load实现原理（度小满）

MySQL Load和Stream Load功能相似, 都是导入本地文件到Doris集群中, 因此MySQL Load实现复用了StreamLoad的基础导入能力:

（1）FE接收到客户端执行的MySQL Load请求, 完成SQL解析工作。

（2）FE将Load请求拆解,并封装为StreamLoad的请求。

（3）FE选择一个BE节点发送StreamLoad请求。

（4）发送请求的同时, FE会异步且流式的从MySQL客户端读取本地文件数据, 并实时的发送到StreamLoad的HTTP请求中。

（5）MySQL客户端数据传输完毕, FE等待StreamLoad完成, 并展示导入成功或者失败的信息给客户端。

2.7 Doris-connector 数据一致性的实现

数据一致性三种：at-most-once; at-least-once; exactly-once。flink提供checkpoint的机制来实现exactly-once，checkpoint机制会为对应流计算过程中的所有的算子制作状态快照。保存在持久化存储中，当任务失败时，flink便会从对应的快照信息中恢复最近一次的完整信息，从而保证数据被精准一次消费。

而Doris flink connector的精准一次依赖于2pc 的机制来实现的。具体流程如下：

- 首先Flink任务启动后便会发起一个stream load的parper请求，此时会先开启一个事务，同事通过http的chunked机制往doris中写入数据
- 在checkpoint时，结束数据的写入，同事完成http的请求，事务状态会变更为precommitted，此时数据已经写入be，但对用户不可见
- checkpnint完成后，发起commit请求，并且更改事务状态为commited，完成后数据对用户可见
- 如果过程中Flink任务意外挂掉，从ckeckpoint重启时，若上次的事务状态为precommitted状态，则会触发回滚请求，并且设置事务状态为aborted

2.8 介绍下Doris数据导入的compaction机制（杭州织光映画科技）

Doris的存储引擎通过类似LSM的数据结构提供快速的数据导入支持。对于单一的数据分片（Tablet），新的数据先写入内存结构，随后刷入磁盘，形成一个个不可变更的数据文件，这些数据文件保存在一个rowset中。而Doris的Compaction机制主要负责根据一定的策略对这些Rowset进行合并，将小文件合并成大文件，进而提升查询性能。

每一个rowset都对应一个版本信息，表示当前rowset的版本范围，版本信息中包含了两个字段first和second，first表示当前rowset的起始版本（start version），second表示当前rowset的结束版本（end version），如图1-1所示。每一次数据导入都会生成一个新的数据版本，保存在一个rowset中。未发生过compaction的rowset的版本信息中first字段和second字段相等；

执行compaction时，相邻的多个rowset会进行合并，生成一个版本范围更大的rowset，合并生成的rowset的版本信息会包含合并前的所有rowset的版本信息。

Compaction分为两种类型：base compaction和cumulative compaction。其中cumulative compaction则主要负责将多个最新导入的rowset合并成较大的rowset，而base compaction会将cumulative compaction产生的rowset合入到start version为0的基线数据版本（Base Rowset）中，是一种开销较大的compaction操作。这两种compaction的边界通过cumulative point来确定。base compaction会将cumulative point之前的所有rowset进行合并，cumulative compaction会在cumulative point之后选择相邻的数个rowset进行合并。

2.9 什么是Doris的单副本导入机制?

在Doris进行数据导入时，首先将数据写入与Tablet相对应的MemTable中。当MemTable写满时，会将数据刷写（Flush）到磁盘。刷写数据到磁盘的过程分为两个阶段：第一阶段将MemTable中的行存结构转换为列存结构，并生成每列的索引结构；第二阶段将转换后的列存结构写入磁盘，生成Segment文件。

但是这样在多副本导入数据的时候就会需要在每个副本上执行MemTable的处理过程，从而使得每个副本所在的节点都会消耗一定的内存和CPU资源，而且这种重复的处理也会降低执行效率。

为了解决这个问题，Apache Doris在2.0版本中引入了单副本导入的功能。该功能是通过从多个副本中选择一个主副本，然后只对主副本进行计算，一旦主副本的数据文件全部写入成功，系统会通知从副本所在节点直接拉取主副本的数据文件，实现了副本间的数据同步。当所有从副本节点拉取完成后，再返回成功或超时返回（大多数副本成功即返回成功）。这种方式在很大程度上减轻了其他节点的负担，降低了计算资源和内存的使用，提高了整体系统的并发吞吐能力。

2.10 介绍下Doris Flink Connector 2PC 实现原理

Doris 的Flink Connector 2PC 的实现是将FLink 2PC 和 Doris Stream Load 2PC 进行结合实现的。首先，分别介绍下这两个内容。

（1）FLink 2PC的实现分为预提交阶段和提交阶段。在预提交阶段，Flink 会在数据流中注入检查点分界线（Checkpoint Barrier），将数据流划分为不同的检查点。每个算子会对当前的状态做一个快照，并保存到状态后端。Sink 端会以事务的方式写数据到外部系统，比如 Kafka，并将事务 ID 保存到状态后端。在提交阶段，当检查点完成时，Flink 会通知所有的 Sink 算子提交事务，将数据写入到外部系统中。如果发生故障或崩溃，Flink 会从最近一次的检查点恢复，并根据事务 ID 回滚或重新提交事务，以保证数据不丢不重。

（2）Doris Stream Load 2PC的实现分为Prepare阶段和触发Commit阶段。在Prepare阶段，在 Prepare 阶段，Coordinator BE 节点会向 FE 发送 Begin Transaction 请求，FE 会开启一个事务，并返回 Transaction Id 给 Coordinator BE。然后，Coordinator BE 会向 FE 发送获取导入计划的请求，FE 会为 Stream Load 任务生成导入计划，并返回给 Coordinator BE。接着，Coordinator BE 会执行导入计划，将数据通过 BRPC 分发到其他 Executor BE 节点。Executor BE 节点会将数据写入存储层，并执行 Prepare Transaction 将对应 Tablet 的数据写入事务添加到 TxnManager 中进行管理。在 Commit 阶段，Executor BE 节点完成数据写入之后，Coordinator BE 节点会向 FE 发送 Commit Transaction 请求，FE 会对事务进行提交，并向 Executor BE 发送 Publish Version 的任务，同时等待 Executor BE 执行 Publish Version 完成。Executor BE 节点会异步执行 Publish Version，将数据导入生成的 Rowset 变为可见数据版本，并执行 Commit Transaction 将数据导入生成的新的 Rowset 添加到 TxnManager 中进行管理。Publish Version 正常完成或执行超时之后，FE 向 Coordinator BE 返回 Commit Transaction 和 Publish Version 的结果。Coordinator BE 向用户返回 Stream Load 的最终结果。

（3）Doris Flink Connector 2PC 在内存中维护读写的 Buffer，在启动的时候，开启写入，并异步的提交，期间通过 HTTP Chunked 的方式持续的将数据写入到 BE，直到 Checkpoint 的时候，停止写入，这样做的好处是避免用户频繁提交 HTTP 带来的开销，Checkpoint 完成后会开启下一阶段的写入。

然后是在这个 Checkpoint 期间，可能是多个 Task 任务同时在写一张表的数据，这些都会在这个 Checkpoint 期间对应一个全局的 Label，在 Checkpoint 的时候将这个 Label 对应的写入数据的事务进行统一的一次提交，将数据状态变成可见。

如果失败 Flink 在重启的时候会对这些数据通过 Checkpoint 进行回放。这样就可以实现了两阶段提交，保证 Doris 两端数据的一致。

2.11 Doris如何实现表结构的变更

表结构的变更在OLAP场景下是一个相对常见的业务需求。一般来说，当上游数据开始变化时候，就需要将对应表结构进行改变。而Doris这块也是经历了不同实现形式的schame change。在Doris 1.2.0 版本中，引入了Light Schema Change 这一新功能，用于改善以往 Hard Linked Schema Change 在处理加减列操作时遇到的一些问题。

先看Hard Linked Schema Change的实现原理：Hard Linked Schema Change 在作用于加减 Value 列时，当接收到加减 Value 列的 DDL，Doris FE（下文简称 FE）会发起一个异步的作业，并立刻返回。作业主要做以下事情：

- 创建新的 Tablet。

- 等待已经开始的导入完成。
- 将当前已有的数据文件 Hard Link 到新 Tablet 对应的数据目录下。

在使用过程中， Hard Linked Schema Change 存在几个明显的问题：

- 当集群规模和表数据量达到一定数量时，Hard Linked Schema Change 等待时间就会明显增加。
- 在 Hard Linked Schema Change 过程中，如果有导入任务，为保证 Schema Change 的事务性，会对新/旧 Tablet 进行双写，Schema Change 则需要等待导入任务完成之后才可以进行。
- Hard Linked Schema Change 无法处理 Delete Predicate。如果用户在 Schema Change 之前调用过 Delete 语句，Doris 不会立刻删除数据，而是记一个Rowset，并把 Delete Predicate 和此 Rowset 进行关联；如果在做 Hard Linked Schema Change 过程中，发现 Delete Predicate，则会转化为 Sort/Direct Schema Change，对数据进行重写。

而Doris 1.2.0 新版本的 Light Schema Change 的实现原理就要简单的多：只需要在加减 Value 列的时候，对 FE 中表的元数据进行修改并持久化，并不需要同步给BE，BE 对读写操作依赖于自身的 Schema。不过这会带来Schema不一致的问题，因此通过修改BE的读写流程来解决：

- 读取数据的时候下发 Schema 。Schema 每一列都有相应的 Unique ID，该 Unique ID 由 BE 的每个 Tablet 负责产生和赋值，但对于所有的 Tablet，其 Schema 列的 Unique ID 都是一致的，因此将此过程移在 FE 上去实现。FE 在生成查询计划时，会把最新的 Schema 附在其中，并一起发给 BE，BE 会拿最新的 Schema 读取数据。
- 将 Schema 持久化到 Rowset 的元数据中。FE 在发起导入任务的时候，会把最新的 Schema 一起下发给 BE，BE 会根据最新的 Schema 对数据进行写入并与 Rowset 绑定，将该 Schema 持久化到 Rowset 的元数据之中，实现了 Rowset 数据的自解析
- 在进行 Compaction 的时候，选取需要进行 Compaction 的 Rowset 中最新的 Schema，作为Compaction 之后 Rowset 所对应的 Schema
- 此外，实现了一个全局的 Schema Cache 管理相同的 Schema，这样就算有成千上万个 Rowset，只要 Schema 相同，内存中只会存在一份 Schema。

2.12 Doris如何进行数据导出?

在目前Doris版本中，可以使用EXPORT进行数据导出，EXPORT支持导出 Doris本地表 / View视图 / 外表，支持导出到 parquet / orc / csv / csv_with_names / csv_with_names_and_types 文件格式。

用户提交一个 Export 作业后。Doris 会统计这个作业涉及的所有 Tablet。然后根据`parallelism`参数（由用户指定）对这些 Tablet 进行分组。每个线程负责一组tablets，生成若干个`SELECT INTO OUTFILE`查询计划。该查询计划会读取所包含的 Tablet 上的数据，然后通过 S3协议 / HDFS协议 / Broker 将数据写到远端存储指定的路径中。

总体的执行流程如下:

（1）用户提交一个 Export 作业到 FE。

（2）FE会统计要导出的所有Tablets，然后根据`parallelism`参数将所有Tablets分组，每一组再根据`maximum_number_of_export_partitions`参数生成若干个`SELECT INTO OUTFILE`查询计划

（3）根据`parallelism`参数，生成相同个数的`ExportTaskExecutor`，每一个`ExportTaskExecutor`由一个线程负责，线程由FE的Job 调度框架去调度执行。

（4）FE的Job调度器会去调度`ExportTaskExecutor`并执行，每一个`ExportTaskExecutor`会串行地去执行由它负责的若干个`SELECT INTO OUTFILE`查询计划。

注：也可以直接使用mysqldump工具导出数据或者表结构

2.13 介绍下Doris中的Sequence列

为了解决Uniq模型中主键唯一性约束使用REPLACE聚合方式，使得在同一批次中导入的数据，替换顺序不做保证以至于无法确定最终导入到表中的具体数据的问题，Doris支持了sequence列，通过用户在导入时指定sequence列，相同key列下，REPLACE聚合类型的列将按照sequence列的值进行替换，较大值可以替换较小值，反之则无法替换。该方法将顺序的确定交给了用户，由用户控制替换顺序。其通过增加一个隐藏列`__DORIS_SEQUENCE_COL__`实现，该列的类型由用户在建表时指定，在导入时确定该列具体值，并依据该值对REPLACE列进行替换。在读取数据时，请求包含value列时需要额外读取`__DORIS_SEQUENCE_COL__`列，该列用于在相同key列下，REPLACE聚合函数替换顺序的依据，较大值可以替换较小值，反之则不能替换。

2.14 Doris/StarRocks 中的列存储和行存储有何区别及应用场景?

StarRocks 主要使用列存储，它与行存储有本质的区别，适用于不同的应用场景：

1.列存储： StarRocks 中的列存储意味着表中的每一列数据分别存储在物理空间上。这种方式使得在进行大规模数据的聚合查询时，能够只读取必要的列数据，减少I/O操作，特别适合分析型查询。

2.行存储： 在行存储模型中，一行的数据是连续存储的。这种方式在需要频繁更新记录或者执行事务处理时比较高效，因为它可以快速访问完整的记录。

3.应用场景对比： 列存储适合于读密集型的场景，尤其是需要大量聚合和分析的查询操作；而行存储则更适合于写密集型的场景，如OLTP（在线事务处理）系统。

2.15 doris明细模型的分区的更新是怎么做的？我要回溯一下、数据更新一下的话，是先删再写吗？（面试题）

Doris 明细模型无主键约束，不支持行级 UPSERT，更新能力有限。分区更新与回溯逻辑如下：

- 分区更新：仅支持 “增量追加” 或 “全量替换”。增量直接追加新数据；全量替换需先删除目标分区旧数据，再导入修改后的数据，可通过临时分区实现原子替换。从而实现分区数据更新
- 数据回溯：按时间分区可直接查询历史分区；或通过备份数据重新导入目标分区完成回溯。

2.16 在doris/starrocks，如何实现数据的全量和增量导入?

在 Doris/StarRocks 中，数据全量与增量导入可通过以下方式实现，兼顾高效性与灵活性：

(1)全量导入

用于初始数据加载或全量覆盖场景，支持多种工具和方式：

- **Broker Load**：通过 HDFS/S3 等存储系统全量导入，适合 TB 级离线数据。步骤：提交导入作业 → 解析文件 → 批量导入至目标表。
- **Spark Load**：基于 Spark 分布式能力，处理 PB 级数据。流程：Spark 读取数据源 → 转换为 Doris 格式 → 批量导入。
- **Insert Into**：SQL 方式逐行 / 批量插入，适合小批量全量更新，如 `INSERT INTO table SELECT * FROM source_table`。
- **Stream Load**：HTTP 协议推模式，适合 GB 级以下实时全量文件（如 CSV/JSON），支持原子性导入。

(2)增量导入

用于数据实时 / 准实时更新，保障数据连续性：

- **Routine Load**：对接 Kafka 等消息队列，实时消费增量数据。配置：创建导入作业 → 持续消费 Topic → 按规则解析并导入目标表（支持数据转换、过滤）。
- **增量 CDC 同步**：基于 MySQL Binlog 等变更日志，通过 Flink/Canal 等工具捕获增量，转换后导入 Doris/StarRocks（需目标表按时间 / 主键分区，保障更新幂等性）。
- **Insert Into 增量**：结合业务逻辑（如时间分区），通过 `INSERT INTO table PARTITION(...)` 增量插入新分区数据。

2.17 讲下Doris表关联物理关联的几种方式（跨越速运/深圳艾特尔/杭州织光映画科技/上海蓝标/metshort）

Doris 的 JOIN 物理执行有以下几种数据 Shuffle 方式：

1. **Broadcast Join**：将右表全量数据广播到所有参与 JOIN 的节点，左表数据不动。网络开销 = N × T(右表)，适用范围广但右表不能太大。
2. **Partition Shuffle Join**：左右表数据都按 JOIN 条件列的 Hash 值重新分发到对应节点。网络开销 = T(左表) + T(右表)，适用于通用场景。
3. **Bucket Shuffle Join**：左表数据不动，只将右表数据按左表的分桶分布进行 Shuffle。网络开销 = T(右表)，要求 JOIN 条件包含左表的分桶列，且左表为单分区。
4. **Colocate Join**：左右表数据都不移动，直接在本地做 JOIN。网络开销 = 0，**要求两表属于同一 Colocation Group，分桶列类型和桶数完全一致**。

灵活性依次降低，但性能依次提升（前提是桶数足够多，并行度不受限）。此外，底层实现上支持 **Hash Join**（等值条件）和 **Nested Loop Join**（非等值/笛卡尔积）两种算子。

2.18 doris如何强制使用broadcast？（跨越速运/上海蓝标）

在 Doris 中，可以通过 **Distribute Hint** 强制指定 Broadcast Join，有以下几种方式：

方式一：直接在 JOIN 前加 `[broadcast]`在 JOIN 关键字后、右表名前加上 `[broadcast]`：`SELECT COUNT(*) FROM t1 JOIN [broadcast] t2 ON t1.c1 = t2.c2;`

方式二：配合 Ordered Hint 使用，固定 JOIN 顺序为文本顺序，再指定 Broadcast：`SELECT /*+ ordered */ COUNT(*) FROM t2 JOIN [broadcast] t1 ON t1.c1 = t2.c2;`

方式三：配合 Leading Hint 使用，在控制 JOIN 顺序的同时指定 Broadcast：`SELECT /*+ leading(t1 broadcast t2) */ COUNT(*) FROM t1 JOIN t2 ON t1.c1 = t2.c2;`

验证是否生效

使用 `EXPLAIN SHAPE PLAN` 查看执行计划，确认分发方式变为 `DistributionSpecReplicated`（表示 Broadcast）：`EXPLAIN SHAPE PLAN SELECT COUNT(*) FROM t1 JOIN [broadcast] t2 ON t1.c1 = t2.c2;`

输出中出现 `PhysicalDistribute[DistributionSpecReplicated]` 即表示 Broadcast 生效。

典型使用场景

当 Join Key 存在**数据倾斜**时，Partition Shuffle 会导致某个节点处理大量数据，此时强制 Broadcast 可以有效避免倾斜问题。

2.19介绍下 Doris/Starrocks SQL 解析流程

Doris sql解析是指将一条sql语句经过一系列的解析最后生成一个完整的物理执行计划的过程。其包括**词法分析，语法分析，生成逻辑计划**和**生成物理计划**四个步骤。

（1）词法分析：主要负责将字符串形式的sql识别成一个个token，为语法分析做准备。

（2）语法分析：主要负责根据语法规则，将词法分析生成的token转成抽象语法树（AST）

（3）逻辑计划：负责将抽象语法树转成代数关系。代数关系是一棵算子树，每个节点代表一种对数据的计算方式，整棵树代表了数据的计算方式以及流动方向。

（4）物理计划：指的是在逻辑计划的基础上，根据机器的分布，数据的分布，决定去哪些机器上执行哪些计算操作。

2.20 实时项目中kafka到doris同步方式是怎样的？ kafka到doris同步 有几种方式？（趣致）

Kafka 到 Doris 的同步方式主要有以下 **3 种**：

1. **Routine Load（推荐首选）**：Doris 内置功能，无需外部组件，直接通过 SQL 创建持久化消费任务，自动拉取 Kafka 数据写入 Doris。支持 CSV 和 JSON 格式，具备 Exactly-Once 语义，秒级延迟，自动故障恢复。
2. **Doris Kafka Connector**：基于 Kafka Connect 框架的插件，以"推"的方式将 Kafka 数据写入 Doris。支持 JSON、Avro、Protobuf 等更多格式，以及 Debezium CDC 数据解析，适合需要处理上游数据库变更数据的场景。
3. **Flink Doris Connector**：通过 Flink 消费 Kafka 数据，再写入 Doris Sink，底层使用 Stream Load。适合需要在写入前做复杂流处理（如 Join、聚合、清洗）的场景，支持 insert/update/delete 事件同步。

实时项目的同步方式，结合自己的项目回答就行。

2.21 公司有用doris的部分列更新吗？怎么使用的/适用什么场景（深圳万邦速达）

一、适用场景

场景	说明
实时高频字段更新	用户标签表实时更新行为字段，供广告/推荐系统使用
多源宽表拼接	多张源表按主键拼接成宽表，每张源表只贡献部分列
数据修正	批量修正某些记录的部分字段，其他字段保持不变

二、两种数据模型的对比

数据模型	写入性能	查询性能	适用场景
Unique Key（MoW）	中（受 IO 影响）	高	实时更新、查询性能敏感
Aggregate Key（REPLACE_IF_NOT_NULL）	高（与普通导入相当）	较低（慢 5-10 倍）	写入吞吐敏感、查询性能可接受

三、Unique Key 模型的使用方式

前提：建表时必须开启 Merge-on-Write，且**必须包含所有 Key 列**。

1. Stream Load

```
curl --location-trusted -u root: \ -H "partial_columns:true" \ -H "column_separator:," \
-H "columns:order_id,order_status" \ -T /tmp/update.csv \
http://127.0.0.1:8030/api/db1/order_tbl/_stream_load
```

2. INSERT INTO

```
SET enable_unique_key_partial_update=true; INSERT INTO order_tbl (order_id, order_status)
VALUES (1, '待发货');
```

3. Flink Connector

```
'sink.properties.partial_columns' = 'true'
```

四、Aggregate Key 模型的使用方式

建表时将需要更新的列设置为 `REPLACE_IF_NOT_NULL`，写入时**无需额外参数，**注意：`REPLACE_IF_NOT_NULL` 不能将字段值改为 NULL。

郑冬综合2502 2026/9/16 06:01:38
涤生大数据版权所有，请勿外传

无标签